

SDD 1003

Projet final (40 points)

Consignes :

Note : Vous pouvez réaliser le projet **individuellement** ou en groupe de **deux personnes maximum**. Il est impératif de préciser quelle partie du projet a été réalisée par chaque membre. En cas d'absence de cette précision, **une note de zéro** sera attribuée à l'ensemble du projet pour les **deux membres**.

Note : Pendant la présentation, vous devez être capable d'expliquer clairement :

- le code,
- les concepts et les algorithmes utilisés,
- ainsi que les résultats obtenus.

Dans le cas contraire, **une note de zéro** sera attribuée pour **l'ensemble du projet**

Note : Pour votre projet final, créez un site web en utilisant Node.js (JavaScript) et MongoDB, ou bien Flask (Python) et MongoDB. Utilisez la base de données que vous avez téléchargée sur Kaggle et déjà téléversée sur Atlas.

Note: Les images suivantes sont uniquement pour vous donner une idée de la fonctionnalité de votre site web. Vous êtes libres de choisir le design et la manière de concevoir votre site web.

Note : Vous êtes libre d'utiliser les modèles ('Template') HTML, JavaScript ou python comme point de départ pour votre projet.

a. Les fonctionnalités attendues de votre site web:

1- Recherche classique (5 points) :

- L'utilisateur doit pouvoir effectuer une recherche sur votre site, comme nous l'avons pratiqué dans l'exemple du cours et illustré dans l'image suivante. Adaptez cet exemple à votre base de données.



Movie Title Search

Search Movie Title:

Scarface



Data loaded successfully. Start typing to search!

Objectif :

Selon votre préférence de design (HTML/CSS/JS ou framework), et en utilisant la connexion à la base de données que vous avez configurée dans **MongoDB Atlas**, vous devez créer une **zone de recherche (text-box)** qui respecte les conditions suivantes :

Spécifications :

- **Connexion à MongoDB Atlas**
 - a. Utilisez la collection que vous avez trouvée sur Kaggle et importée dans MongoDB Atlas (par exemple, movies).

- **Comportement de la zone de recherche :**
 - a. **Cas a)** Si l'utilisateur ne saisit rien et appuie sur la barre d'espace, les 10 premiers titres doivent s'afficher automatiquement.
 - b. **Cas b)** Si l'utilisateur commence à taper un mot, les premiers titres correspondants doivent apparaître automatiquement (**auto-complétion**).
 - c. Une fois qu'un mot est choisi dans les étapes a et b, la page doit mettre à jour les informations.
- **Affichage :**
 - a. Affichez les titres dans une liste sous la zone de recherche.
 - b. **Ajoutez un message comme :**
« Les données ont été chargées avec succès. Commencez à taper pour rechercher ! » lorsque la connexion est établie.
 - c. Vous devez afficher au moins 5 champs pour chaque document correspondant à la recherche (par exemple : titre, année, réalisateur, genre, résumé).

2- Créer, lire, mettre à jour et supprimer (CRUD) (5 points) :

Une fois le document trouvé à l'étape précédente, le logiciel doit permettre à l'utilisateur d'effectuer les opérations CRUD : créer, lire, mettre à jour et supprimer:

- d. Par exemple, l'utilisateur doit pouvoir modifier un champ dans un document, puis cliquer sur le bouton « Mettre à jour ». L'application doit alors mettre à jour automatiquement le document dans la base de données et afficher son contenu.
- **Remarque :** Le design des boutons CRUD fait partie de votre projet et dépend de votre créativité. Ils doivent être esthétiques et ergonomiques.

3- Intégration de la recherche vectorielle avec MongoDB et le Machine Learning

Partie 1 : Comprendre la recherche vectorielle

Avant de commencer, assurez-vous de bien comprendre ce qu'est la **recherche vectorielle**, c'est-à-dire une méthode qui permet de retrouver des

documents similaires en comparant leurs **représentations vectorielles** (embeddings).

- **Objectif** : Intégrer une requête vectorielle dans MongoDB.
Exemple : Retrouver les films sémantiquement proches d'un film donné, comme *Interstellar*.
-

Partie 2 : Utilisation du Machine Learning (15 points)

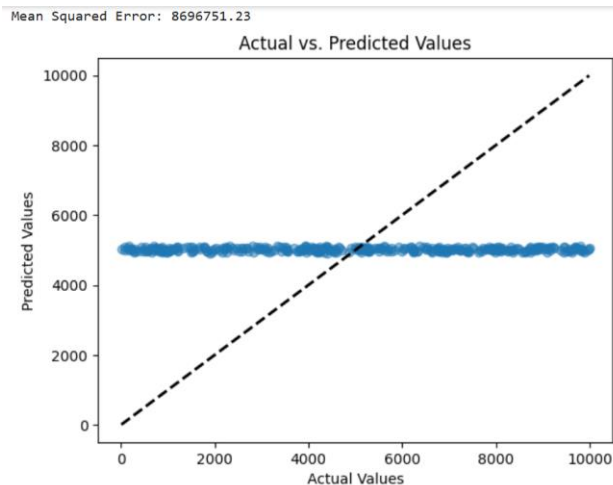
Vous devez ensuite utiliser un **modèle d'apprentissage automatique** pour transformer les descriptions des films dans votre base de données en **vecteurs numériques**.

Étapes à suivre pour intégrer la recherche vectorielle dans MongoDB :

1. **Génération des embeddings**
Utilisez un modèle de machine learning (par exemple, un modèle de type *Sentence Transformers*) pour transformer chaque description en vecteur.
2. **Stockage dans MongoDB (1 point)**
Ajoutez les vecteurs générés comme champs dans vos documents MongoDB.
3. **Indexation vectorielle (1 point)**
Créez un index vectoriel dans MongoDB pour permettre la recherche par similarité.
4. **Recherche vectorielle (1 point)**
Ajoutez un nouveau champ de saisie intitulé « Recherche vectorielle de films ». À partir de la requête saisie (description d'un film ou mot-clé), générez l'embedding correspondant, puis effectuez une recherche de similarité dans MongoDB afin d'identifier les films les plus proches dans l'espace vectoriel.
5. **Post-traitement (6 points)**
Appliquez trois modèles de **classement ML** pour classer les résultats obtenus. **Exemples** :

- **Random Forest** ou **Logistic Regression** pour classifier les films selon leur genre (ex. : ne garder que les films de science-fiction).
- **XGBoost** ou **Régression linéaire** pour prédire un score de pertinence basé sur des variables comme la note IMDb ou la popularité.
- **K-Means** pour regrouper les films par thématique et ne conserver que ceux du même cluster que *Interstellar*.

Vous devez visualiser les résultats des algorithmes de classement et de regroupement présentés ci-dessus à l'aide de graphiques clairs. L'exemple suivant est fourni à titre informatif:



- Votre code doit également effectuer la catégorisation des films (ou éléments de la base) en appliquant les trois algorithmes mentionnés ci-dessus. **(6 points)**

4- Analyse statistique avec la fonction de distribution empirique (EDF) sur votre base de données MongoDB (5 points)

Tâches à réaliser :

1. Connexion à MongoDB Atlas

Utilisez **Node.js** ou **Python (Flask)** pour interroger votre base de données.

2. Choisir une variable quantitative

Exemples : rating, runtime, ou year.

3. Calculer l'EDF

L'EDF est définie comme :

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{X_i \leq x}$$

où n est le nombre total d'observations.

4. Visualisation

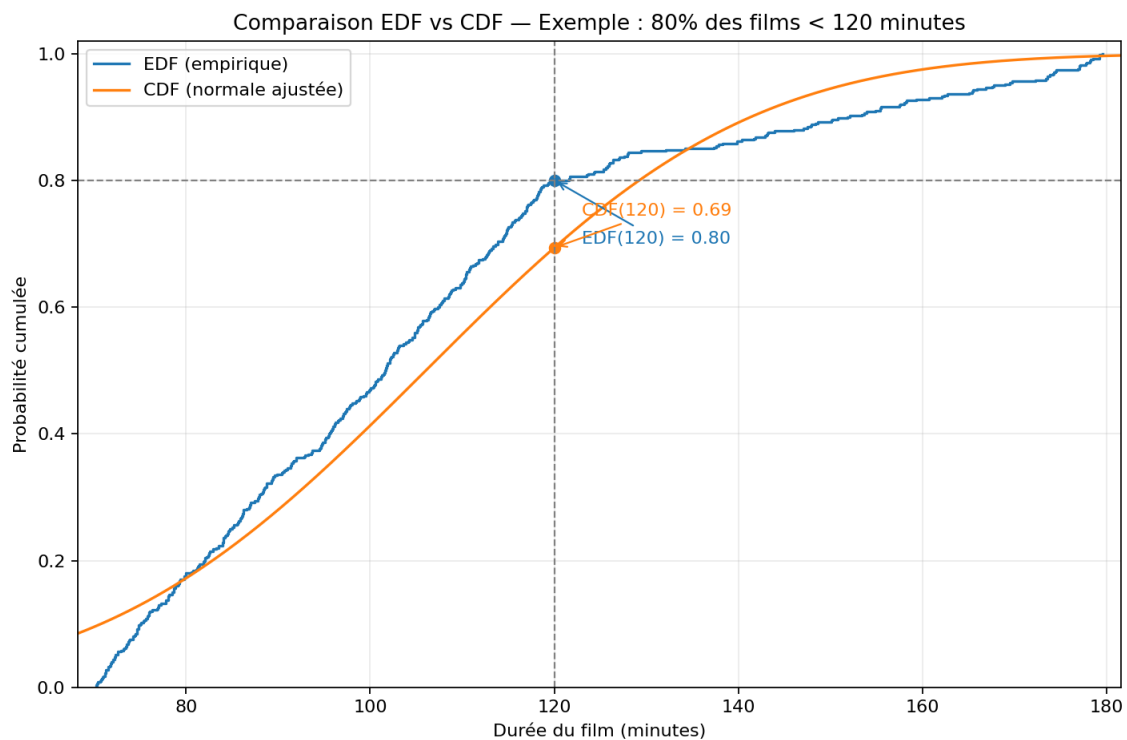
Tracez les courbes EDF et CDF pour votre variable en utilisant Matplotlib, Seaborn ou toute autre bibliothèque de votre choix.

- **CDF** : fonction théorique (ex. : loi normale).
- **EDF** : approximation empirique basée sur vos données réelles.

5. Interprétation

Analysez la distribution des films.

Exemple : « 80 % des films ont une durée inférieure à 120 minutes »



d) Veuillez créer un fichier zip contenant votre code avec des commentaires, et le téléverser sur le PORTAIL de COURS. Ce fichier zip doit contenir la liste des bibliothèques nécessaires à l'exécution du projet. Lorsqu'il est utilisé avec la commande `pip install -r requirements.txt`, toutes les dépendances doivent être installées automatiquement, permettant ainsi de lancer votre code sans configuration supplémentaire (5 points).

Note : Veuillez vous assurer que la commande `pip install -r requirements.txt` fonctionne correctement. Si je ne peux pas exécuter votre code à cause d'un problème d'installation des dépendances, une note de zéro sera attribuée à l'ensemble du projet.

e) Vous devez présenter votre code le **11 décembre 2025** en utilisant la commande `pip install -r requirements.txt` (5 points).

Note : Si vous avez choisi de travailler en équipe de deux, le jour de la présentation, les deux membres du groupe doivent être présents. En cas d'absence, une note de zéro sera attribuée pour le projet.